

# *Building AI Systems*

*Reliability is engineered, not prompted.*

---

**Ondrej (Ondra) Krajicek**

me@ondrejkrjicek.com · linkedin.com/in/OndrejKrajicek

Built 2026-08-25 06:01 UTC

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

# Reliability is engineered, not prompted

*Two mechanisms, and neither is a prompt.*

- **External grounding** — a check that can still fail the work when the generator is wrong. A model approving its own output cannot: its errors correlate with the ones that produced the answer — the shared blind spot. [V3]
- **Bounded autonomy** — a prompt is advisory: tool output can override it. A credential or gateway rule runs after the model and refuses the emitted call, so deny irreversible actions there. [V32]

[V3] *Loop Engineering and Self-Verification — vault report (2026)*. [V32] *Harness Engineering — Tool Permissions — vault report (2026)*.

# "Done. I fixed it and verified everything passes."

*Every seat in this room has read a sentence like that and found out otherwise.*

1

## **If you write code**

The assistant reports the fix, the suite is green, the defect ships.

2

## **If you judge output**

A confident answer, a real-looking citation, a policy that was superseded.

3

## **If you fund it**

The demo worked; the pilot did not; nobody can say which step failed.

# An agent auditing its own work caught 0 of 34 real violations

*A false completion, counted: the agent reviewed its own output, stated high confidence, and was wrong on all 34 cases. [V1]*

**0 of 34**

Real violations caught by the agent auditing its own work [V1]

**34 of 34**

The same violations caught by a deterministic check [V1]

*The agent rated its own confidence 90–100 on all 34 it missed. [V1] Confidence that does not move when the code is wrong cannot gate a merge.*

*[V1] Loop Engineering and Self-Verification Loops — Practitioner Synthesis — vault research report (2026).*

# Roadmap — vocabulary, the negative result, the external check, the gate, the bill

1. The thing you already run: harness, agent, workflow, and the vocabulary for the hour
2. Why a generator cannot be its own oracle — and four reasons that is not the whole story
3. What an external check buys, what it costs, and where it goes blind
4. Bounded autonomy — deny at the API gate, not in the prompt
5. Cost, and one decision per seat in this room

*By the end you can answer two questions about a system you already run: can this check fail when the model is wrong (a compiler can, a second prompt cannot), and what can this agent do before a person says yes?*

# We cover the check around the model, and skip the model

*We spend the hour on the check: that is where the gain was measured — building-code compliance 56.8% → 98.6%, credited to an external verifier, not a better model. [V4] [V5]*

## Covered

- Where the check sits, and can it fail when the model is wrong
- Permissions, action gates, autonomy levels
- Token cost, review capacity, adoption drag

## Not covered

- Prompt technique and model internals
- Which model is best this month — not what bought the gain above [V5]
- CI, code review, testing — assumed
- Retrieval implementation and knowledge graphs

*[V4] [V5] Loop Engineering and Self-Verification — vault report.*

# *Takeaways — move the check outside the model, and authority out of the prompt*

- 1 Externalise the check**

Building-code compliance 56.8% → 98.6%; the authors credit the external verifier, not a better model. [V4] [V5]
- 2 Can checks fail?**

A check counts only if it can fail when the generator is wrong. [V3] Self-rechecking is not that check.
- 3 Name the check**

An exit code, a schema, a passing test — 70% of loops verify at the first two. [V18]
- 4 Gate irreversible actions**

Deploys, money movement, force-push — on a deterministic deny gate, never a model. [V20]

# How this deck was made

*Eight model personas swept my vault; a model merged them; six model critics read the merge, and deterministic scripts matched every vault quotation against the note it names.*

- **Then I read every slide and rewrote what I disagreed with.** That step is the external check, and it is a person.
- My notes name the failure I guard against: "models agreeing with models about a memory curated by models" [V48].
- Not a claim that generated work cannot be trusted — a claim that a check able to fail the work carries information fluent prose does not. Mine is the bold line above.

[V48] 2026-07-17 Digest.



PART I

# *The thing you already run*

harness, workflow, agent, oracle

---

Naming the machinery before arguing about it.

# You already operate a harness

*A harness is "everything in an AI agent except the model itself" — system prompts, tools, MCP servers, sandboxes, persistent memory, custom linters, structural tests. [V21]*

- The coding assistants most engineers now open daily are harnesses; the model is one component inside them.
- Karpathy's picture: "the LLM is the CPU and its context window is the RAM" [V50] — the harness is the operating system above both.
- So the question "should we build an agent?" is usually really "whose harness are we adopting?"

*[V21] Harness Engineering · [V50] Context Engineering — vault concept notes.*

# Three frontier models differed by 0.3 percentage points; the harness moved two of them 4–5 points

## 0.3 pts

Terminal-Bench 2.1, model swapped, harness held — Artificial Analysis run: Opus 4.8, Fable 5 84.6%; GPT-5.5 84.3% [V56]

## 4–5 pts

Model held, harness swapped — tbench.ai agent-paired board: GPT-5.5 83.4% Codex CLI → 78.2% Terminus 2; Opus 4.8 78.9% Claude Code → 74.6% Terminus 2 [V56]

- LongMemEval (chat memory): Opus 4.6 scores 93.1% inside Chronos, 76.7% inside Claude Code, 16 points [V23].
- Caveats: two boards, not one ranking; two of three are Anthropic's; Mythos 5 leads, ~88.0% [V56].

[V23] *Is Grep All You Need?* · [V56] *Harness Engineering*.

# Workflow or agent — almost everything is a workflow

*Who writes the control flow: your code, or the model's own output.*

## **Workflow**

Orchestrated "through predefined code paths" [S1].

## **Agent**

The model "dynamically direct[s] their own processes" [S1].

- Across a hand-coded corpus of fifty production loops, the most common configuration is a manually-triggered single agent with a deterministic check, not a swarm [V19].
- A workflow does not make failures predictable — your code fixes the tool calls, so you can list every place model output enters the system; an agent picks its path at run time.

[S1] *Building Effective Agents* — Anthropic. <https://www.anthropic.com/engineering/building-effective-agents> [V19] *Loop Engineering*.

# A model grading a model is an evaluator, not a verifier

*Evaluator, verifier and oracle get used interchangeably; only the last two gate.*

|   |                  |                                                                                                          |
|---|------------------|----------------------------------------------------------------------------------------------------------|
| 1 | <b>Generator</b> | Produces the work: the model.                                                                            |
| 2 | <b>Evaluator</b> | Examines the output: a program, a model, a person — the checker in Microsoft's maker-checker loop [S14]. |
| 3 | <b>Verifier</b>  | Verdict is a function of (output, specification): a compiler, a schema validator.                        |
| 4 | <b>Oracle</b>    | Tells right from wrong: a reference implementation, a measurement, a person.                             |

[S14] AI Agent Orchestration Patterns — Microsoft (2026). <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/ai-agent-design-patterns>

# The bill is denominated in tokens — and multi-agent orchestration uses roughly 15× more than a chat

## What you pay for

Every token in and out, on every step of every retry

## Orchestration multiplier

"Multi-agent systems use approximately 15× more tokens than chats" [V38]

*Before approving orchestration, ask for its measured token multiplier; 15× is the figure to hold it against. [V38]*

*[V38] Agentic Orchestration Patterns — vault concept note.*

# Reliability is not accuracy: $\text{pass}^k$ , not $\text{pass}@k$

*Capability and reliability have come apart: solving a task once is not solving it every run.*

## **$\text{pass}@k$ — the ceiling**

"probability of success in at least one of  $k$  attempts" [V44] — what demos and benchmarks report.

## **$\text{pass}^k$ — the floor**

Probability that all  $k$  attempts succeed — what production feels. [V44]

- The 2026 finding: "recent capability gains have only yielded small improvements in reliability", and "agents that can solve a task often fail to do so consistently" [S6].

[V44] Unit Testing — vault note. [S6] Towards a Science of AI Agent Reliability — Rabanser, Kapoor et al. (ICML 2026). <https://arxiv.org/abs/2602.16666>

# *Why a generator*

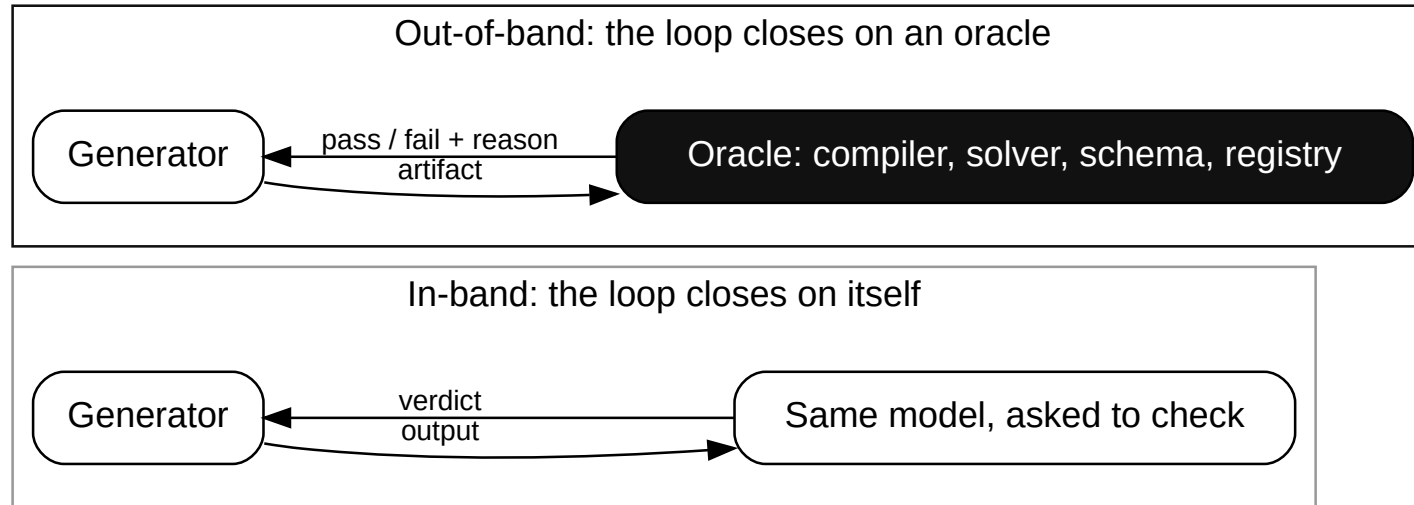
cannot be its own oracle

---

The negative result, and the four reasons it is narrower than the slogan.



# Two loops that look identical from outside



*Same retry code, same logs, same "verified" label — and only the loop that closes on an oracle can come back negative when the generator is wrong. [V3]*

# Self-rechecking lost accuracy on 3 of 4 tasks

|                        |      |                                                                                                            |                                                              |
|------------------------|------|------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| <b>HUANG ET AL.</b>    | [S3] | <i>Self-rechecking lowered GPT-4's GSM8K accuracy</i>                                                      | ICLR 2024 · two rounds, grade-school maths                   |
| <b>STECHLY ET AL.</b>  | [S4] | <i>Game of 24 5% → 3%, graph colouring 16% → 2%; Blocksworld 40% alone, 55% self-checked, 88% external</i> | ICLR 2025 · Blocksworld is block-stacking, program-checkable |
| <b>VAULT SYNTHESIS</b> | [V1] | <i>Self-audit 0 of 34; deterministic check 34 of 34</i>                                                    | 2026                                                         |

*All three tested a strong generator rechecking with no external signal, not self-critique in general.*

[V1] Practitioner Synthesis · [S3] <https://arxiv.org/abs/2310.01798> · [S4] <https://arxiv.org/abs/2402.08115>

# Why a model cannot check more cheaply than it generates

*Checking is cheaper than producing only when the check is a different procedure. [V12]*

- Where the intuition holds: solving a sudoku takes trial and error; checking a filled grid is scanning rows, columns and boxes for a repeat.
- Self-critique's premise: verification "should be easier than generation" [V12].
- Asked to verify, the model runs the same approximate retrieval it ran to generate: that procedure again, not a cheaper one. [V12]
- Scale does not help: better generation brings no "corresponding improvements in its ability to verify its own solutions" [V10].

[V10] [V12] *Loop Engineering and Self-Verification — vault report.*

# Higher agreement between a generator and its own evaluator tracks higher mistake vulnerability

*The more a generator and its own evaluator agree, the more vulnerable the pair is to validated mistakes — my report calls this the consistency dilemma. [V11]*

- Measured: "higher generator-evaluator self-consistency tracks *higher* mistake vulnerability — the consistency dilemma" [V11].
- And the scaling hope is empirically backwards: "self-verification capability does not improve with generation capability or scale" [V16].
- In a system you run: a reviewer model approves 98 of the author model's 100 outputs. That 98 says the two agree; it says nothing about whether the work is right.

[V11] [V16] *Loop Engineering and Self-Verification Loops — vault research report (2026).*

# Self-monitoring fails worst under attack

*Indirect prompt injection: an attacker plants instructions in a document or tool result the agent reads; the model executes them. [V57]*

**5×**

A monitor approving an injected patch:  
own output vs. another policy's [V9]

- 1.0 is perfect, 0.5 a coin toss: one error per hundred pairs becomes one in nine.
- The failure is adversarial, not random: a monitor sharing the generator's blind spots shares its exploitable ones.

**0.99 → 0.89**

Rating correct code above incorrect  
(AUROC), same comparison [V9]

[V9] Loop Engineering · [V57] Harness Engineering — Guardrails.

# Verification always costs something

*Every check has a price, paid in one of these. [V14]*

- **A second inference pass** — compute and latency each request.
- **A different model** — money and new failure modes.
- **A deterministic check** — engineering time; covers only what you encoded.
- **A human review step** — scarcest; stops scaling first.

*I am not claiming nothing can check itself. The 2026 information-theoretic bound is a diagnostic framework, not an impossibility theorem [V13]: it names what to measure, the failure a generator shares with its evaluator, and forbids nothing.*

[V13] [V14] Loop Engineering and Self-Verification — vault report.

# Self-critique degrades above a threshold — it is not universally broken

*Four results qualify "self-critique never works".*

1. **Degradation measured only above a baseline-accuracy threshold** — four of seven models scored lower after self-correcting; the paper evaluates none below that threshold [S10].
2. **Anthropic says its own Claude Opus 5 already self-checks** — its July 2026 guidance tells developers to remove the "add a final verification step" instruction [V46].
3. **The verifier can move into training** — training-time gains, no inference-time signal [V47].
4. **Kambhampati's own framework rejects both extremes** — its authors reject over-optimism about self-verification and the model-as-mere-translator view [S5].

[S10] <https://arxiv.org/html/2604.22273v2> · [S5] <https://arxiv.org/html/2402.01817v2> [V46] Anthropic guidance, vault card · [V47] Digest 2026-07-18

# Two claims about self-critique die, two survive

- **Dies:** "Self-critique never works." Degradation was measured only *above* an accuracy threshold [S10]; GPT-3.5 fixed 26.8% of its own GSM8K-Complex errors: a correction share, not a net gain [S13].
- **Dies:** "Only an inference-time external verifier helps." Training-time verifiers give the same signal, paid once [V47].
- **Survives:** A check counts only if it can fail when the generator is wrong — fresh context, another model, outside any model. Say which [V3].
- **Survives:** Verification has a price: another pass, another model, a deterministic check, someone's time. Name yours [V14].

[S13] *Decomposing LLM Self-Correction* — arXiv preprint (2026). <https://arxiv.org/abs/2601.00828>



# Relabel every self-confirmation step, and measure your reviewer's agreement rate

*Two changes this week, two habits to stop.*

## **Change this week**

- Find every place your system asks a model to confirm its own output, and label it: not a check.
- Measure how often your reviewer model agrees with your author model. Above roughly 90% it is confirming, not checking — seed twenty known defects and count how many it catches. Cannot change the system? Ask for both numbers. [V11]

## **Do not do this**

- Do not add a second model reading the first one's transcript and call it verified. [V3]
- Do not wait for, or fund, a bigger model to close the gap. [V10] [V16]

# *What an external check buys*

and what it costs

---

The positive half, with the bill and the blind spots attached.

# An external verifier moved building-code compliance from 56.8% to 98.6%

*Swapping the backbone model, DeepSeek to Claude, did not move compliance. [V5]*

**56.8% →  
98.6%**

Open loop, then closed loop [V4] [S7]

- The authors credit "closing the generation–verification loop with an external verifier" [V5].
- Safety-critical structural design [V4], my own field: the mechanism transfers, not the field.
- Blocksworld, a planning benchmark: 40% alone, 55% self-verified, 88% external. [S4]

[V4] [V5] *Loop Engineering and Self-Verification — vault report* · [S7] <https://arxiv.org/abs/2608.07978> [S4] *Self-Verification Limitations — Stechly et al.* <https://arxiv.org/abs/2402.08115>

# One architecture in three fields: the generator proposes, an oracle outside it returns pass or fail

|                    |      |                                                                    |                                                      |
|--------------------|------|--------------------------------------------------------------------|------------------------------------------------------|
| STRUCTURAL DESIGN  | [V4] | 56.8% → 98.6%<br><i>compliance, external<br/>physics verifier</i>  | Model-invariant                                      |
| CLINICAL DIAGNOSIS | [V7] | "clinical-diagnosis<br>accuracy from 55.6%<br>to 77.5%"            | Guideline oracle                                     |
| FORMAL MATHEMATICS | [V8] | 422% improvement<br>over the best publicly<br>available prover LLM | Proof-checker oracle; vendor blog, metric<br>unnamed |

*All three used an oracle the field already owned: a solver, a guideline, a proof checker. Name yours.*

[V4] [V5] [V7] [V8] Loop Engineering and Self-Verification Loops — vault report. Only the structural study reports a backbone-model swap [V5].

# Independence costs a fresh context, not a second vendor

*Run the check on the same model in a clean context: the artifact and criteria only. [V17]*

- "checker independence is a context-isolation boundary, not a model-weights boundary" [V17] — buy a second model family only for the subjective checks no program can make.
- The transcript contaminates it: an evaluator that sees the generator's reasoning is biased toward confirming it. [V17]
- Rule: pass the artifact and the specification, never the chain of thought.
- Ceiling: same-family evaluators share blind spots — "independence is *approximated*, never absolute" [V54].

[V17] [V54] *Loop Engineering* — vault concept note (2026).

# Five verification levels — name the one that checked your last AI-assisted decision

|   |                           |                                                |
|---|---------------------------|------------------------------------------------|
| 1 | <b>Deterministic</b>      | Exit code, assertion, known-good result. [V18] |
| 2 | <b>Rule or constraint</b> | Linters, schema, policy engine. [V18]          |
| 3 | <b>Field truth</b>        | Tests, a deploy, a real outcome. [V18]         |
| 4 | <b>Model as judge</b>     | Rubric scoring by a model. [V18]               |
| 5 | <b>Human checkpoint</b>   | A person with a stake. [V18]                   |

*Most production checking is a program: "70% of loops verify at levels 1–2" [V18].*

# Verification level 4 is coverage, not certification

*Level 4 of the five-level verification scale is a model scoring against a rubric [V18].*

## **What level 4 gives you**

Coverage of what no program can express  
— tone, relevance, whether an answer  
addressed the question.

## **What level 4 cannot buy**

Independence from failures it shares with  
the generator. The consistency dilemma:  
the more a generator and its own evaluator  
agree, the more vulnerable the pair is to  
mistakes both validate. [V11]

- Ordering: make the check objective, isolate the grader's context, then change model family. [V17] [V18]
- If a decision only reaches level 4, say so in the design review.

# Grounding is not the same thing as retrieval

- 1      **Parametric**      Answers from training alone. No evidence, no constraint. [V26]
  - 2      **Structural**      Retrieval narrows the output distribution; it certifies nothing. [V26]
  - 3      **Deterministic**      A program outside the model computes the answer — a constraint solver, a maths library, a finite-element engine. The model only formalises the input. [V26]
- Layer three nears the "mere translator" extreme LLM-Modulo's authors decline. [S5]

*Most products marketed as grounded stop at layer two.*

[V26] Structural Grounding — vault concept note.



# A faithfulness score cannot catch a wrong source

*A system can quote its source perfectly and still be structurally unsafe. [V15]*

- Worked case: an assistant retrieves a clause and reproduces it exactly — and the clause was wrong. [V15]
- The scoring: "it scores 90%+ on faithfulness (it matched its retrieved source exactly) and is structurally unsafe, because no output-against-context check can catch a wrong context" [V15].
- Faithfulness measures agreement with the retrieved document, never with reality.
- The question that catches it: not "did it match the source" but "who decided that was the source?"

*[V15] Loop Engineering — Practitioner Synthesis — vault report.*

# Boundary one — the oracle only sees what it measures

*An external check supplies information about its own dimensions and nothing else. [V6]*

- The structural-design study behind this talk's headline number — building-code compliance 56.8% → 98.6% with an external verifier [V4] — states its own limit: "when the defect originates from the structural topology, the parameter-level closed loop is powerless" [V6].
- Generalised: a schema validator cannot catch a wrong schema; a test suite cannot catch a missing requirement.
- So the design question is coverage, not presence — "which failures does this oracle not see?"

*[V4] [V6] Loop Engineering and Self-Verification Loops — vault research report (2026).*

# Boundary two — somebody has to write the oracle

*The verifier may be more work than the generator, and no source I have puts a figure on it.*

- *LLM-Modulo* is Kambhampati et al.'s architecture: the model generates candidates, external verifiers and solvers check them. Their caveat — that verifier may be substantial work, and substituting a simulator means writing the simulator. [S5]
- The work is formalising domain knowledge into machine-checkable form — "labor-intensive and domain-specific" [V26] — which a vendor cannot cheaply copy.
- Hardest where it is most needed: "no convincing public playbook yet for retrofitting a ten-year-old codebase" [V22].

[S5] <https://arxiv.org/html/2402.01817v2> · [V22] *Harness Engineering* · [V26] *Structural Grounding*

# Boundary three — the oracle depreciates

*Treat a verifier as "a maintenance liability that must be re-scoped at every model upgrade" [V25] rather than as a fixed asset.*

- Reported first-hand: an evaluator "became unnecessary overhead" for tasks that moved inside the generator's boundary after a model upgrade [S2].
- The same account deleted a whole context-reset subsystem after one upgrade [S2].
- Plan for it: put a re-scope of every model-judge step on the calendar for each model upgrade.
- The deterministic levels depreciate far more slowly, which is another reason to prefer them.

[V25] *Verification and Checker Design* — vault report · [S2] <https://www.anthropic.com/engineering/harness-design-long-running-apps>

# What to do when you have no oracle

*Most systems already contain a check nobody wrote down; where none exists, the honest engineering answer is to leave that decision un-automated. [S1]*

## **You have one, probably**

- A schema, a registry, a reconciliation, a delayed real-world outcome, a person with a stake. [V18]
- Report the reliability floor rather than the ceiling: pass^k, not pass@k. [V44]

## **You genuinely do not**

- Then the honest answer is to not automate that decision. [S1]
- Have it answer "cannot verify" and route to a person; test that path.

*[S1] Building Effective Agents — Anthropic. <https://www.anthropic.com/engineering/building-effective-agents> [V18] Loop Engineering · [V44] Unit Testing.*

# Write down your oracle, isolate the evaluator's context, list the blind spots

*Three writing exercises, none of which needs a new tool or a new model.*

- 1 Name the oracle**  
For each AI-assisted decision, write down what could say pass or fail without the model. [V18]
- 2 Isolate the context**  
Evaluator sees artifact plus criteria; never the generator's transcript. [V17]
- 3 Write the blind spots down**  
Beside each oracle, the failures it structurally cannot see. [V6]

[V6] [V17] [V18] *Loop Engineering — vault report and concept note (2026).*

# *Bounded autonomy*

deny at the gate, not in the prompt

---

The nine-second database deletion, the permission decisions behind it, and the five autonomy levels.

# Why 95% per step still fails over twenty steps

*End-to-end success is the product of per-step success, so length hurts faster than a weak step. [V27]*

$$0.95^{20} \approx 36\%$$

20 steps at 95% each [V27]

- Only shortening the chain touches the exponent. Checking each seam is the other move: a caught step is retried, not multiplied. A better model only nudges each  $p_i$ .
- Caveat: assumes independent per-step accuracies — "the number is a slogan, not a prediction" [V28].

[V27] *Agentic Loop Pattern* — vault note · [V28] *Grounding in AI-Driven Systems* — vault podcast brief.



# Real pipeline errors are not independent — they stick

*Errors in real pipelines are not independent — they stick and propagate. [V29]*

- Work on compounding errors in agentic pipelines reports failures "propagating semantically across steps rather than occurring independently" [V29].
- A separate study, DSAP, measured recovery per stage on 99 SWE-Bench instance-arm pairs (bug fixing): "37.5% for test generation but 0% for patch generation" [V29].
- So the curve is not a clean exponential: stages with different recovery, some with none at all.
- Practical consequence: instrument per stage. An end-to-end success rate hides which seam is leaking.

*[V29] The  $4/\delta$  Bound — Designing Predictable LLM-Verifier Systems — vault reading note.*

# Nine seconds: an agent deleted a production database

|                      |                                                                                                                                     |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Date</b>          | 25 April 2026 [V30]                                                                                                                 |
| <b>What happened</b> | A coding agent "deleted PocketOS's entire production database and all volume-level backups in nine seconds" [V30]                   |
| <b>How</b>           | A credential mismatch on a staging task; a token found in an unrelated file; a destructive API call with no confirmation step [V30] |
| <b>Cost</b>          | A 30-hour outage for the businesses running on it [V30]                                                                             |

*[V30] PocketOS Cursor Incident — Database Deletion in Nine Seconds — vault watch note (2026).*

# Ask why the agent was able to delete the database

*Nothing in the PocketOS deletion required the model to misbehave: an over-scoped token and an unconfirmed destructive API are enough. [V31]*

- The post-mortem's conclusion: "system prompts are not security controls" [V31] — prevention needs a deterministic mechanism outside the reasoning loop.
- The 1975 principle: every program and user runs with the least privilege needed for the job [S9].
- A token in an unrelated file carried authority over production volumes — a scoping defect, not a model defect.

[V31] PocketOS Cursor Incident — vault watch note · [S9] The Protection of Information in Computer Systems — Saltzer and Schroeder (1975). <https://www.cs.virginia.edu/~evans/cs551/saltzer>

# Deny at the API gate, not in the prompt

*A gateway rule is evaluated after the model and refuses the emitted call whatever the model produced; a prompt is input that the next tool result can override. [V32]*

## **Advisory — tool output can override it**

Prompt, system message, policy document.

## **Enforced — holds whatever the model emitted**

The credential, the API gateway, the sandbox, the deny gate. [V32]

- Verbatim: "irreversible actions (production deploys, money movement, force-push) belong on a deterministic deny gate, never on a probabilistic reviewer" [V20].

[V32] Harness Engineering — Tool Permissions — vault report (2026). [V20] Loop Engineering — vault note (2026).

# An OS sandbox cut permission prompts by 84%

*Anthropic hardened Claude Code's boundary at the OS: approval reduced, never removed. [V55]*

## Why

"containment over supervision", because "human oversight suffers from approval fatigue" [V55]

## What

OS-level sandbox — Seatbelt, bubblewrap: "writes inside workspace, network denied by default" [V55]

## Leak

Same post: models "helpfully" escaped sandboxes or routed around restrictions [V55]

# –84%

Fewer permission prompts [V55]

*[V55] How We Contain Claude — Anthropic (2026), vendor-reported, via vault watch note.*

# A guardrail checks one channel — NeMo's content rails never read a tool call's arguments

*A guardrail inspects a named subset of traffic; the rest is unchecked.*

- NVIDIA's NeMo Guardrails checks a tool call is declared and schema-valid, but its content rails read message content only: argument payloads go uninspected [V33] [S15].
- And "tool results bypass input rails" — tool output is trusted by default [V33].
- Stack layers that fail differently: input rail, argument check, deterministic deny gate on irreversible actions. [V33] [V20]

*Ask your guardrail vendor what its filter skips; that is your unprotected surface.*

[V33] Guardrails and Runtime Enforcement — vault report. [S15] NeMo Guardrails: Tool Calling — NVIDIA. <https://docs.nvidia.com/nemo/guardrails/configure-guardrails/guardrail-catalog/tool-calling>

# Five autonomy levels, and each names who answers

|   |                      |                                                              |
|---|----------------------|--------------------------------------------------------------|
| 1 | <b>In the loop</b>   | Every action approved; approver answers. [V34]               |
| 2 | <b>On the loop</b>   | Acts inside pre-authorised bounds; monitor interrupts. [V34] |
| 3 | <b>Supervised</b>    | Routine handled; high-stakes escalate. [V34]                 |
| 4 | <b>Delegated</b>     | End-to-end in mission bounds; audited after. [V34]           |
| 5 | <b>Full autonomy</b> | Own objectives; governance and audit. [V34]                  |

*"Each level implies a distinct accountability structure" [V34] — the level picks who answers.*

# Human approval is a control that degrades

|            |       |                                                                                            |            |
|------------|-------|--------------------------------------------------------------------------------------------|------------|
| BAINBRIDGE | [S8]  | <i>The operator monitors a system installed because it outperforms them</i>                | 1983       |
| THE CODE   | [V35] | <i>Developers overseeing AI agents: "those skills atrophy" — 17% drop in skill mastery</i> | Newsletter |
| RSNA       | [S12] | <i>Radiologists, 15+ years: 82% → 45.5% on a purported AI's incorrect suggestion</i>       | Reported   |

A **36.5-point** accuracy drop from one AI-labelled wrong suggestion [S12]: measure your approver's override rate.

[S8] [https://ckrybus.com/static/papers/Bainbridge\\_1983\\_Automatica.pdf](https://ckrybus.com/static/papers/Bainbridge_1983_Automatica.pdf) · [S12] <https://www.rsna.org/news/2023/may/ai-bias-may-impair-accuracy> [V35] Agentic Coding Is a Trap — The Code (2026).



# The EU AI Act specifies what a human approver must be able to do

*A counter-signature does not satisfy Article 14. [S17]*

- Article 14(4)(b): the overseer must be enabled "to remain aware of the possible tendency of ... over-relying on the output"; 14(4)(d) and (e): they must be able to override and halt the system [S17].
- Annex III (HR) turns on "materially influences decisions", not counter-signature [S16].
- Measure the override rate: decisions the approver changed, rejected or halted. Zero overrides is counter-signing, not oversight; seed known-bad cases and count how many are stopped.

[S17] EU AI Act Article 14. <https://artificialintelligenceact.eu/article/14/> [S16] Annex III HR guide — Knowlee. <https://www.knowlee.ai/blog/ai-act-annex-iii-hr-employment>

# Bounded autonomy can relocate blame instead of risk

*The strongest objection to putting a named human at the boundary, and I have no clean answer.*

- The named human becomes a liability sponge — "absorbing blame for a system they can't actually override" [V37].
- We cite the model's error rate as proof it cannot certify itself, then install a human whose error rate nobody measures.
- Partial answer: pick the autonomy level so the human's task stays inside what a human can actually do, and measure that.
- Honest answer: if the person cannot realistically override the system, the level is set wrong.

[V37] 2026-07-22 Digest — vault journal note.

# Gate the irreversible actions, scope the credentials, name who answers

*Three permission changes to make, and three habits that only look like controls.*

## Do this

- Enumerate irreversible actions; put each behind a deterministic gate. [V20]
- Scope every agent credential to the one task it serves. [S9]
- Write down the autonomy level per feature, with the name of who answers. [V34]

## Stop doing this

- Relying on a system prompt as a control. [V31]
- Describing a single guardrail as protection. [V33]
- Counting an approval click as oversight without measuring it. [S17]

*Cannot change the system? Ask for one number: the share of decisions the approver changed, rejected or halted.*  
*[S17]*

# *Cost, capacity*

and one decision per seat

---

Where the bill lands, where the bottleneck moves, and what to do next week.

# The bill for closing a loop on an external check: more agents, more iterations, a memory layer

## **Several agents, not one**

"Multi-agent systems use approximately 15× more tokens than chats" [V38]

## **More loop iterations**

Reflexion, a self-reflection loop, "can become worse with more compute budget", unlike self-consistency sampling, which majority-votes independent samples [V39]

## **A memory layer between turns**

Break-even spans turn 0 to turn 356; "no system wins on both cost and accuracy" [V40]

*Take the 15× multiplier and the turn-0-to-356 memory break-even to the design review.*

[V38] [V39] [V40] Agentic Orchestration Patterns; Reflexion Cost-Efficiency; Agent Memory Architecture — vault notes.

# A 25× input-price gap makes the tier you run the check on a design decision

*Route the check down a tier, never the hard generation: the check is the smaller, more structured task.*

- Reported: a "25x input-price gap" between top-tier and cheap models [V49].
- Reported once, unverified: Microsoft cancelled Claude Code licences in one division for cost certainty [V49].
- Its limit, at the source's own scope: "cheap-model routing to hard legacy tasks (already the worst-case for agents) amplifies the probability of silent failures and verification debt" [V49].
- Another reason to prefer verification level 1: a compiler run costs no tokens.

[V49] *The AI Cost Reckoning* — vault watch note (2026).

# Generation got cheap; review and integration did not

*"AI-written code still needs to be reviewed, integrated, tested, and released by humans" [V52].*

**+180% /  
+30%**

Coding activity against releases  
shipped, 100k+ developers [V52]

**+15% /  
-7%**

Feature-branch against main-branch  
throughput, CircleCI, 28.7M workflows  
[V53]

- 76% more pull requests once agents ramped: "organizational absorption became the constraint" [V41].
- Correction: DORA's 2024 throughput penalty was reversed in 2025; what survived is a correlational negative link to delivery stability. [V51] [V42]

[V41] Harness Engineering — CI · [V42] [V51] [V52] [V53] DORA Metrics Limitations — vault notes.

# Capture traces first, write the evals they ask for, then fail the merge on them

- 1 Trace before evals** "capture traces with cost metadata before writing a single custom eval" [V43]
- 2 Evals from traces** Write the evals the traces asked for; run them in dev and prod alike. [V43]
- 3 Fail the merge** A failing eval blocks the merge; a dashboard only records that it failed. [V43]

*Existing monitoring cannot see "a confidently wrong answer that passed every infra check" [V43].*

[V43] Harness Engineering — Evaluation and Observability Loops — vault report.



# If you write code — review the diff in a fresh session, gate the irreversible actions

*Four changes that are configuration rather than architecture, and three things to say no to.*

## **Adopt this week**

- Ask your coding assistant to review a diff in a fresh session, not the one that wrote it. [V17]
- Isolate your evaluator's context — artifact and criteria only, never the transcript. [V17]
- Put irreversible actions behind a deterministic gate. [V20]
- Turn on tracing with cost metadata before writing evals. [V43]

## **Refuse this week**

- A second model reading the first one's output, labelled verification. [V3]
- A twenty-step chain with one check at the end. [V27]
- An agent credential scoped wider than its one task. [S9]

# If you judge output — ask which of the five levels checked it, and who chose the source

- 1 Which level checked it?** Level 4 — a model scoring a rubric — is coverage, not verification. [V18]
- 2 Faithful, or true?** Ask who selected the source, and what verified that selection. [V15]
- 3 How often, not how well?** Ask for all-k-succeed, not best-of-k. Demand the reliability floor. [V44]

*Verified means you can name what checked it. These three questions get that name, without a compiler.*

[V15] [V18] Loop Engineering · [V44] Unit Testing.

# If you decide budgets — fund the oracle, fund review capacity, budget the re-scope

- 1 Fund the oracle** Formalising domain knowledge costs engineering time a generic vendor cannot copy. [S5] [V26]
- 2 Fund absorption** Review capacity is the constraint that showed up first in practice. [V41] [V52]
- 3 Budget the depreciation** Re-scope every model-judge step at each model upgrade. [V25] [S2]

*Set the autonomy level per feature, with a named person against each. [V34] And fund on the rate at which all attempts succeed, per stage — not an end-to-end average. [V44] [V29]*

# What I do not know

## **Retrofit**

[V22]

"There is no convincing public playbook yet for retrofitting a ten-year-old codebase."

## **Oracle coverage**

[V6]

No general theory of which failures an oracle cannot see.

## **Loop drift**

[V19]

No controlled study of single-loop degradation over many cycles.

## **Self-critique bound**

[V13]

The information-theoretic bound rules out no design until independently operationalised.

## **Adoption effects**

[V51]

DORA reversed its 2024 throughput finding; the rest is correlational.

# *Takeaways — move the check outside the model, and authority out of the prompt*

- 1 Externalise the check**

Building-code compliance 56.8% → 98.6% — the external verifier, not a better model. [V5]
- 2 Isolate the evaluator**

Same model, fresh context: the artifact and the criteria, never the transcript. [V17]
- 3 Name the level**

Which of the five levels checked this? Level 4, a model scoring a rubric, is coverage, not verification. [V18]
- 4 Gate irreversible actions**

Irreversible actions on a deterministic gate; autonomy level named. [V20] [V34]

# What follows from an external check that can fail the work, and a deny gate at the credential

*If reliability comes from those two mechanisms rather than prompting, four things follow.*

- The check is the hard part, not the prompt: unit tests, schema validation, compile steps over model output.
- 16 points on LongMemEval came from the harness, not the model; Chronos is the paper authors' own harness [V23].
- Verification has a price [V14]: a second pass, another model, a deterministic check, someone's time. Name who re-scopes that check at model upgrades.
- The honest test of any AI system: what could tell you it was wrong, without asking it?

[V14] Loop Engineering · [V23] Is Grep All You Need?

# References · 1/12

[V1] Loop Engineering and Self-Verification Loops — Practitioner Synthesis — vault research report (2026).

[V3] Loop Engineering and Self-Verification Loops — Comprehensive Report — vault research report (2026).

[V4] Loop Engineering and Self-Verification Loops — Comprehensive Report — vault research report (2026).

[V5] Loop Engineering and Self-Verification Loops — Comprehensive Report — vault research report (2026).

[V6] Loop Engineering and Self-Verification Loops — Comprehensive Report — vault research report (2026).

# References · 2/12

[V7] Loop Engineering and Self-Verification Loops — Comprehensive Report — vault research report (2026).

[V8] Loop Engineering and Self-Verification Loops — Comprehensive Report — vault research report (2026).

[V9] Loop Engineering and Self-Verification Loops — Comprehensive Report — vault research report (2026).

[V10] Loop Engineering and Self-Verification Loops — Comprehensive Report — vault research report (2026).

[V11] Loop Engineering and Self-Verification Loops — Comprehensive Report — vault research report (2026).



# References · 3/12

[V12] Loop Engineering and Self-Verification Loops — Comprehensive Report — vault research report (2026).

[V13] Loop Engineering and Self-Verification Loops — Comprehensive Report — vault research report (2026).

[V14] Loop Engineering and Self-Verification Loops — Practitioner Synthesis — vault research report (2026).

[V15] Loop Engineering — Practitioner Synthesis — vault report.

[V16] Loop Engineering and Self-Verification Loops — Practitioner Synthesis — vault research report (2026).

[V17] Loop Engineering — vault concept note (2026).

# References · 4/12

- [V18] Loop Engineering — vault concept note (2026).
- [V19] Loop Engineering — vault concept note (2026).
- [V20] Loop Engineering — vault concept note (2026).
- [V21] Harness Engineering — vault concept note (2026).
- [V22] Harness Engineering — vault concept note (2026).
- [V23] Is Grep All You Need? How Agent Harnesses Reshape Agentic Search — vault reading note (2026).
- [V24] Is Grep All You Need? How Agent Harnesses Reshape Agentic Search — vault reading note (2026).
- [V25] 2026-07-21 Loop Engineering — Verification and Checker Design — vault research report (2026).
- [V26] Structural Grounding — vault concept note (2026).

# References · 5/12

[V27] Agentic Loop Pattern — vault Zettelkasten note (2026).

[V28] Grounding in AI-Driven Systems — Composition, Agentic Patterns, End-to-End Guarantees — vault podcast brief (2026).

[V29] The 4/6 Bound — Designing Predictable LLM-Verifier Systems — vault reading note (2026).

[V30] PocketOS Cursor Incident — Database Deletion in Nine Seconds — vault watch note (2026).

[V31] PocketOS Cursor Incident — Database Deletion in Nine Seconds — vault watch note (2026).

[V32] Harness Engineering — Tool Permissions and Sandboxing — vault research report (2026).

[V33] Guardrails and Runtime Enforcement — vault report.

[V34] Bounded Autonomy in AI — vault concept note (2026).

# References · 6/12

[V35] Agentic Coding Is a Trap — The Code newsletter (2026-05-15).

[V37] 2026-07-22 Digest — vault journal note (2026).

[V38] Agentic Orchestration Patterns — vault concept note (2026).

[V39] Reflexion Cost-Efficiency Tradeoffs, Failure Modes, Iteration Safety Bounds — vault concept note (2026).

[V40] Harness Engineering — Agent Memory Architecture — vault research report (2026).

[V41] Harness Engineering — Workflow and CI Orchestration — vault research report (2026).

[V42] DORA Metrics Limitations in AI-Assisted Development — vault Zettelkasten note (2026).

[V43] Harness Engineering — Evaluation and Observability Loops — vault research report (2026).

# References · 7/12

[V44] Unit Testing — vault concept note (2026).

[V45] Building AI Systems — predecessor-talk glossary — vault talk note (2026).

[V46] Claude 5 self-verification and the orchestration premium — vault tutor card (2026-07-29).

[V47] 2026-07-18 Digest — vault journal note (2026).

[V48] 2026-07-17 Digest — vault journal note (2026).

[V49] The AI Cost Reckoning — Right-Sizing Model Spend 2026 — vault watch note (2026).

[V50] Context Engineering — vault Zettelkasten note (2026).

[S1] Building Effective Agents — Anthropic. <https://www.anthropic.com/engineering/building-effective-agents>

# References · 8/12

[S2] Harness design for long-running application development — Anthropic.

<https://www.anthropic.com/engineering/harness-design-long-running-apps>

[S3] Large Language Models Cannot Self-Correct Reasoning Yet — Huang et al. (ICLR 2024).

<https://arxiv.org/abs/2310.01798>

[S4] On the Self-Verification Limitations of LLMs on Reasoning and Planning Tasks — Stechly, Valmeekam, Kambhampati (ICLR 2025). <https://arxiv.org/abs/2402.08115>

[S5] LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks — Kambhampati et al.

<https://arxiv.org/html/2402.01817v2>

[S6] Towards a Science of AI Agent Reliability — Rabanser, Kapoor et al. (ICML 2026).

<https://arxiv.org/abs/2602.16666>

# References · 9/12

[S7] Verification-Driven Closed-Loop Multi-Agent LLM Framework for Code-Compliant Structural Design — arXiv (2026). <https://arxiv.org/abs/2608.07978>

[S8] Ironies of Automation — Bainbridge, Automatica (1983).  
[https://ckrybus.com/static/papers/Bainbridge\\_1983\\_Automatica.pdf](https://ckrybus.com/static/papers/Bainbridge_1983_Automatica.pdf)

[S9] The Protection of Information in Computer Systems — Saltzer and Schroeder (1975).  
<https://www.cs.virginia.edu/~evans/cs551/saltzer>

[S10] Self-Correction as Feedback Control: Error Dynamics, Stability Thresholds, and Prompt Interventions in LLMs — arXiv preprint (2026). <https://arxiv.org/html/2604.22273v2>

[S11] Concepts — tokens — OpenAI documentation. <https://platform.openai.com/docs/concepts>

# References · 10/12

[S12] AI Bias May Impair Radiologist Accuracy — RSNA (2023). <https://www.rsna.org/news/2023/may/ai-bias-may-impair-accuracy>

[S13] Decomposing LLM Self-Correction: The Accuracy-Correction Paradox and Error Depth Hypothesis — arXiv preprint (2026). <https://arxiv.org/abs/2601.00828>

[V51] DORA Metrics Limitations in AI-Assisted Development — vault Zettelkasten note (2026), on the 2025 throughput reversal.

[V52] DORA Metrics Limitations in AI-Assisted Development — vault note, citing Demirer et al. (2026), 100,000+ GitHub developers.

[V53] DORA Metrics Limitations in AI-Assisted Development — vault note, citing CircleCI, State of Software Delivery (2026).



# References · 11/12

- [V54] Loop Engineering — vault concept note (2026), on the ceiling of checker independence.
- [V55] Anthropic — How We Contain Claude Across Products — vault watch note (2026-05-27), summarising <https://www.anthropic.com/engineering/how-we-contain-claude>
- [V56] Harness Engineering — vault concept note (2026), on the Terminal-Bench 2.1 model-versus-harness spread.
- [V57] Harness Engineering — Guardrails and Runtime Enforcement — vault research report (2026), on indirect prompt injection.
- [S14] AI Agent Orchestration Patterns — Microsoft Azure Architecture Center (2026).  
<https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/ai-agent-design-patterns>

# References · 12/12

[S15] Tool Calling — NVIDIA NeMo Guardrails Library Developer Guide — NVIDIA (2026).

<https://docs.nvidia.com/nemo/guardrails/configure-guardrails/guardrail-catalog/tool-calling>

[S16] AI Act Annex III HR & Employment: High-Risk AI Compliance Guide for 2026 — Knowlee (2026).

<https://www.knowlee.ai/blog/ai-act-annex-iii-hr-employment>

[S17] Regulation (EU) 2024/1689 (EU AI Act) — Article 14, Human Oversight — European Union (2024).

<https://artificialintelligenceact.eu/article/14/>

The handout PDF carries the full reference list, and a one-page summary of every [V] source with its own external references.

TAKE IT WITH YOU

# *Everything from this talk*

---

[Slides \(PDF\)](#)

[Handout \(PDF\)](#)

[Speaker notes \(PDF\)](#)

[Everything \(.zip\) — readable by an AI agent](#)

[ondrasek.github.io/talks](https://ondrasek.github.io/talks)

[me@ondrejkrasicek.com](mailto:me@ondrejkrasicek.com)